CS 260: Data Structures

Big O

How to Compare Algorithms?

- Execution time
  - Bad idea
  - Dependent on system
- Number of comparisons
  - Bad idea
  - Dependent on language used and individual style
- Ideal solution – Express the running time of the algorithm as a function of the input size
- Machine-independent
  - Does not depend on
    - Clock speed
    - Hardware
    - Language used

Phone Book Analysis

- Create an algorithm for finding a name from a phone book

Types of Analysis

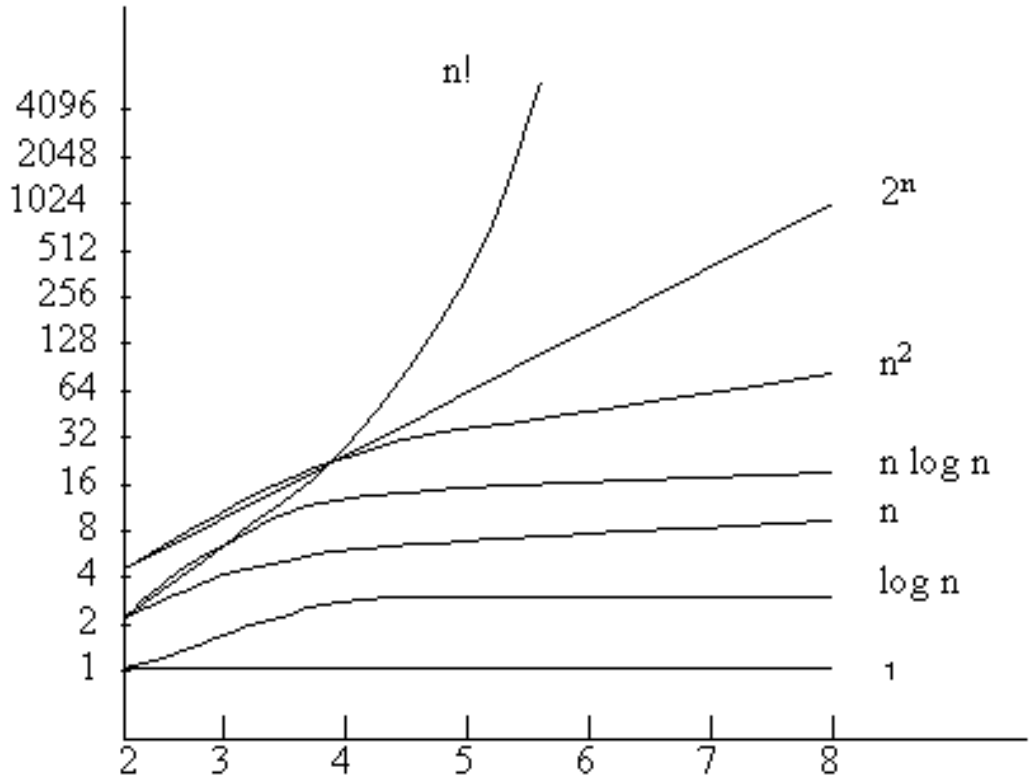
- Worst case
  - Defines the input for the longest execution time
- Best case
  - Defines the input for the shortest execution time
- Average case
  - Prediction about the running time of an algorithm
  - Assume random input

- Lower bound <= Average time <= Upper bound

Big O

- Big O notation is a mathematical notation that describes the limiting behavior of a function when the argument tends towards a particular value or infinity
- Describes change in execution time relative to change in input size
- Consider the function: $f(n) = n^4 + 100n^2 + 10n + 25$
  - This function is $O(n^4)$
  - Generally lower terms of n are discarded
  - Constants are discarded

Growth Rate Chart



Upper Bounds

- We are most concerned with large values of n
- This is generally the most important measure
- For example
  - F(n) = 100log(n)
  - If n = 4, the result is 200. The constant (100) has a large bearing on the result.
  - If n = 1,000,000, the result is 1,993. The input is much larger, but the output is not.

n	f(n)
1	100
128	700
1,024	1,000
10,000	1,320
100,000	1,660
1,000,000	1,993

Big O Examples

- $f(n) = 3n + 7$
  - $3n + 7 = O(n)$
- $f(n) = n^2 + 1$
  - $n^2 + 1 = O(n^2)$
  - Does n^3 provide an upper bound?
    - Not a tight upper bound
- $n^2 + 2 = O(n^2)$
- $2n^2 + 3 = O(n^2)$

More Big O Examples

- f(n) = 20n – n
  - Simplify to 19n
  - Solution: O(n)
- f(n) = 3n * 2n
  - Simplify to $6n^2$
  - Solution: $O(n^2)$

Growth Rate Practical Examples

Time complexity	Name	Example
1	Constant	Add an element to a linked list
Log(n)	Logarithmic	Search a sorted array
n	Linear	Search an unsorted array
nlog(n)	Linear Logarithmic	Sort n items using ‘divide & conquer’
n^2	Quadratic	Bubble sort
n^3	Cubic	Matrix multiplication
2^n	Exponential	Towers of Hanoi problem

Execution Times

- The following table assumes an input of 10,000 with a machine capable of 1,000,000 operations per second.

Function	Common name	Running time
N!	Factorial	
2^n	Exponential	> century
N^d, d > 3	Polynomial	
N^3	Cubic	31.7 years
N^2	Quadratic	2.8 hours
N sqrt n		31.6 seconds
N log n		1.2 seconds
N	Linear	0.1 second
sqrt (n)	Root-n	$3.2 * 10^{-4}$ seconds
Log n	Logarithmic	$1.2 * 10^{-5}$ seconds
1	Constant	

Determine Big O For a Loop

- This loop will execute n times.
- Constant operations inside of the loop.

```
Double min (double data[], int n) {
        Int i;
        Double min = data[0];
        For(i=0;i<n;i++) {
                if(data[i] < min) {
                        min = data[i];
                }//if
        }//for
        Return min;
}//function
```

Non-Trivial Loop

- Some loops are not O(n)

```
void someFunc (int n) {
        Int i;
        for(i=1;i < n;i*2) {
                printf(“result: %d”, i);
        }//for
}//function
```

Nested Loops I

- Loops can de dependent or independent

```
void someOtherFunc (int n) {
    Int i, j;
    for(i=1;i < n;i++) {
        for(j=0;j < n;j++) {
            //constant operations
        }
    }//for
}//function
```

Nested Loops II

- Loops can de dependent or independent

```
void someOtherFunc (int n) {
    Int i, j;
    for(i=1;i < n;i++) {
        for(j=0;j < n;j*2) {
            //constant operations
        }
    }//for
}//function
```

Independent Loops

```
void yetAnotherFunc (int n) {
    Int i, j;
    for(i=1;i < n;i*2) {
        //constant operations
    }//for
    for(j=0;j < n;j++) {
        //constant operations
    }
}//function
```

Big O – Calling Functions I

- When one function calls another, the Big O of the called function is considered.

```
Void sort(int arr[], int n) {
      int i;
      for(i=0;i<n;i++) {
            arr[i] = min(arr, n);   // O(n)
      }//for
}//function
```

Big O – Calling Functions II

- You can think of this as a nested loop if it helps

```
Void otherSort(int arr[], int n) {
        int i;
        for(i=0;i<n;i++) {
                printr("result: %d", search(arr, n); // O(logn)
        }//for
}//function
```

Amortized Analysis

- Big O is the best way to analyze, but it is overly general
- Many algorithms have the same worst case
  - Some are better than others
- Big O is overly pessimistic
- Example
  - Adding elements to a dynamic array
  - Re-size is costly – O(n)
  - Nearly every add is O(1)
  - Often expressed as O(1)+

Big O Practice

- Deescribe the Big O of each code sample

- For(i=0;i < n;i++) {
  - //O(log n) stuff
- }

- For(i=1;i < n;i=i*2) {
  - //constant stuff
- }

Bubble Sort

- Void badbubbleSort(int *arr) {
  - Int max = arr[0];
  - For(i=0;i<arr.length;i;i++) {
    - For(j=0;j<arr.length;j++) {
      - Int temp = arr[j];
      - If(temp > arr[j+1]) {
      - Swap(I, j);
      - Temp = arr[j+1];
      - }
    - }//for
  - }//for
- }//func

Best Case, Worst Case, Average Case?

```
int arrContains(struct dynArr *arr, val x) {
	Int i = 0;
	While(i < arr->size) {
		if(arr->data[i] == x) {
			return 1;
		}
		i++;
	}//while
	return 0;
}//func
```

Best Case, Worst Case, Average Case

```
Void addArr(struct dynArr *arr, TYPE val) {
        if (arr->cap == arr->size) {
                arr->cap *= 2;
                struct stack *new= malloc(sizeof(struct dynArr)*arr->cap);
                assert(new != NULL);
                int i = 0;
                While(i < arr->sz) {
                        new [i] = arr->data[i];
                        i++;
                }//while
                free(arr->data);
                arr->data = new;
        }
        new[i] = val;
        arr->size++;
}//func
```